Software Architecture Models – Part 2

Distributed and interaction-centric paradigms

SEAtS | 641230

University of Southampton

Intended Learning Outcomes

By the end of this lecture you will be able to:

1. Describe five major interaction-oriented software architecture models.
2. Differentiate their communication patterns and structural organisation.
3. Evaluate their strengths and weaknesses in real-world applications.
4. Apply these models to design scalable, maintainable, and resilient systems.

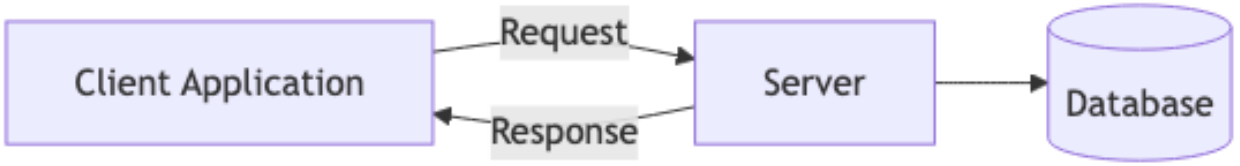
© University of Southampton

Agenda

- Refresh: interaction patterns and concept definitions.
- Deep dives: client-server, SOA, event-driven, MVC, pipe-and-filter.
- Comparative analysis and integration patterns.
- Hybrid use cases and design exercises.

© University of Southampton

Client-Server Model



The **client-server model** underpins networked systems. Clients request resources or services; servers respond with data or actions—forming the backbone of the web, email, and database ecosystems.

© University of Southampton

Client–Server

Advantages

- Clear separation of presentation (client) and core logic (server).
- Centralised control over data, security, and auditing.
- Servers can be updated independently of clients.

Disadvantages

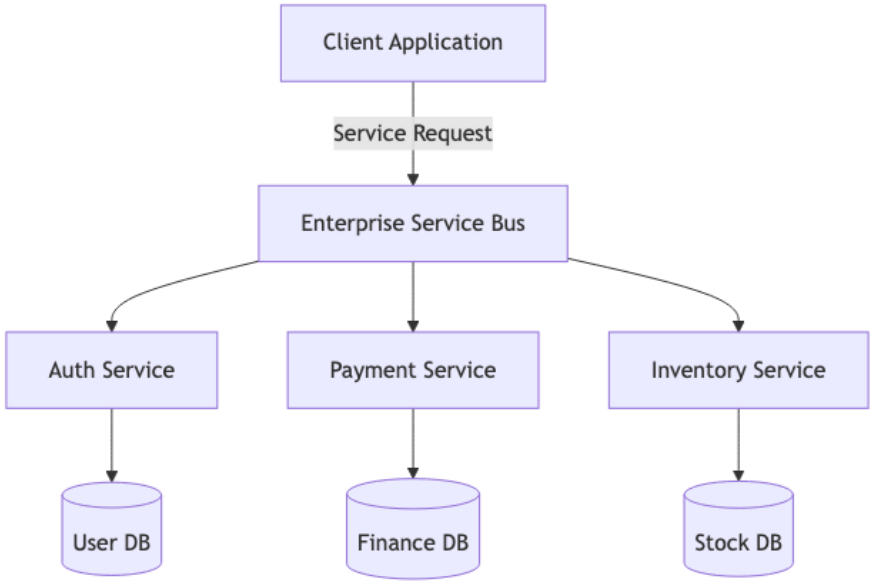
- Single point of failure if the server crashes.
- Scalability constrained by server capacity.
- Network latency impacts responsiveness.

Evolution:

- 2-tier: client ↔ database (desktop apps).
- 3-tier: client ↔ app server ↔ database (web apps).
- N-tier: distributed services and microservices.

© University of Southampton

Service-Oriented Architecture (SOA)



SOA exposes business capabilities as services that communicate through standard interfaces and protocols (e.g., SOAP, WSDL, XML) via an enterprise service bus (ESB).

© University of Southampton

Service-Oriented Architecture (SOA)

Advantages

- Interoperability across platforms.
- Service reuse for business agility.
- Modularity supports scaling by function.

Disadvantages

- Heavyweight ESB and XML overhead.
- Complex governance and versioning.
- Limited fine-grained scalability—precursor to microservices.

Evolution:

- Loose coupling via defined interfaces.
- Services registered for discoverability and reuse.
- Standard protocols (SOAP, XML, UDDI) enable interoperability.

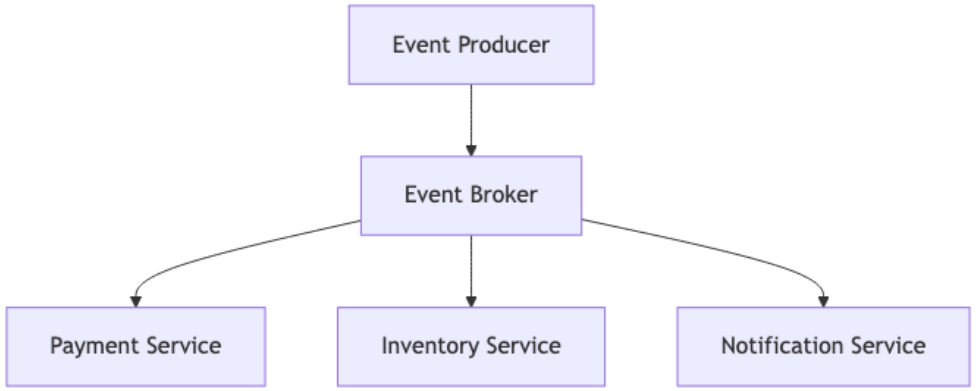
© University of Southampton

Transition to Microservices

SOA introduced modularity but relied on central ESBs. Microservices evolved this concept by decentralising integration, using lightweight APIs, and aligning services with bounded contexts. Understanding SOA provides context for modern microservice design patterns.

© University of Southampton

Event-Driven Architecture (EDA)



In **EDA**, components communicate via events rather than direct requests. Producers emit events; consumers subscribe and react asynchronously—ideal for reactive and real-time systems.

© University of Southampton

Event-Driven Architecture (EDA)

Advantages

- Highly scalable and fault tolerant.
- Enables real-time analytics and reactive behaviour.
- Decouples producers and consumers.

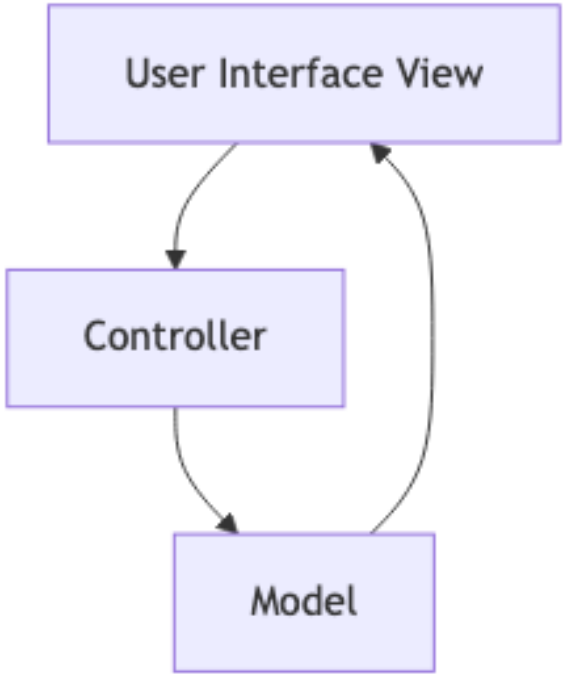
Disadvantages

- Complex debugging and tracing across events.
- Eventual consistency challenges.
- Requires rigorous monitoring and schema governance.

Example domains: IoT, marketplaces, financial monitoring using Kafka, RabbitMQ, AWS SNS/SQS.

© University of Southampton

Model–View–Controller (MVC)



MVC separates presentation, business logic, and data access to simplify UI-centric applications.

© University of Southampton

MVC – Principles & Variants

Advantages:

- Promotes separation of concerns and testability.
- Supports multiple views over shared data models.
- Widely adopted in web and GUI frameworks (Django, Rails, Spring MVC).

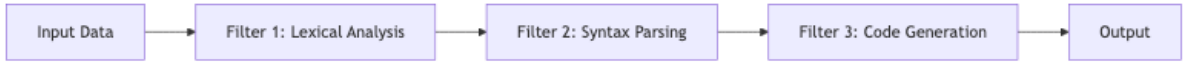
Disadvantages: Controller–View coupling can expand in complex applications, requiring refactoring.

Variants:

- MVVM (Model–View–ViewModel) for data-binding UIs.
- MVP (Model–View–Presenter) for heavily interactive interfaces.

© University of Southampton

Pipe-and-Filter Architecture



The **pipe-and-filter** model structures applications as sequential processing steps (filters) connected by channels (pipes). Each filter transforms input before passing it onward.

© University of Southampton

Pipe-and-Filter – Architecture

Advantages

- Simple, modular processing steps.
- Filters can be reused and replaced independently.
- Fits ETL workflows, compilers, and streaming pipelines.

Disadvantages

- Inefficient for complex branching or backtracking flows.
- Intermediate data storage introduces overhead.
- Handling bidirectional streams is challenging.

© University of Southampton

Comparative Summary

Model	Focus	Typical Domain
Client–Server	Centralised request–response	Internet, databases
Service–Oriented Architecture (SOA)	Modular services on an integration bus	Enterprise systems
Event–Driven Architecture (EDA)	Asynchronous event communication	IoT, real–time processing
Model–View–Controller (MVC)	Separation of presentation, logic, data	Web frameworks
Pipe–and–Filter	Sequential data transformations	Compilers, data pipelines

Model	Communication Type	Coupling	Primary Use
Client–Server	Request–response	Medium	Web, databases
SOA	Message–based via ESB	Low	Enterprise integration
EDA	Publish–subscribe	Very low	Real–time, reactive systems
MVC	Controller–driven updates	Medium–High	Web & GUI apps
Pipe–and–Filter	Sequential data flow	Low	Data pipelines, compilers

© University of Southampton

Architectural Integration

Modern enterprises combine these paradigms to meet diverse requirements:

- Client-server frontends often implement MVC or MVVM.
- Backends expose services via SOA or microservices.
- Internal data processing uses pipe-and-filter pipelines.
- Asynchronous workflows leverage event-driven messaging.

Hybrid architectures allow teams to choose patterns that best fit each subsystem while maintaining coherence.

© University of Southampton

Summary

- Interaction-centric architectures define how components communicate and collaborate.
- Selection depends on scalability, consistency, and operational maturity needs.
- Combining paradigms yields flexible, resilient enterprise solutions.

© University of Southampton

References

- Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice.*
- Papazoglou, M. (2007). *Service-Oriented Architecture: Concepts, Technology, and Design.*
- Chen, L. (2019). Event-driven microservices: lessons learned. *IEEE Software, 36(6).*
- Kruchten, P. (1995). The 4+1 view model of software architecture. *IEEE Software, 12(6).*
- Fowler, M. (2003). *Patterns of Enterprise Application Architecture.*

© University of Southampton